

LABORATORIO 3

Interfaz Gráfica con Processing y Protocolo Firmata

Preparado por:

Luisdavid Colina V24766381

Samantha Ramírez V31307714

1. Desarrollo de actividades

1.1. Actividad 5.1: Carga del firmware StandardFirmata

Enunciado: Compile y suba a la placa el código StandardFirmata disponible en la sección de ejemplos del IDE de Arduino.

Firmata es un protocolo que permite controlar los pines del Arduino desde un programa externo en el computador sin tener que reprogramar la placa cada vez [1]. Para cargarlo, en el IDE se seleccionó *Archivo* → *Ejemplos* → *Firmata* → *StandardFirmata* y se subió a la placa por USB. Con eso el Arduino queda escuchando comandos por el puerto serial y Processing puede comunicarse con él directamente.

1.2. Actividad 5.2: Control de LED RGB desde Processing

Enunciado: Construya una interfaz de usuario, utilizando Processing, que permita controlar la intensidad y el color de un led RGB conectado a la placa, los controles deben ser de tipo slider. Muestre un componente que represente al LED RGB y que modifique su color conforme a los sliders. Implemente un componente que muestre los valores de los componentes RGB y un gráfico de intensidad de color.

Para el circuito se conectaron las tres patas del LED RGB a los pines 3, 6 y 5 de la placa (rojo, verde y azul), cada una con su resistencia de 220 Ω . Se usaron esos pines en particular porque tienen soporte PWM, que es lo que se necesita para variar la intensidad de cada color. El diseño en Fritzing se puede ver en la Figura 1 y el montaje armado en la Figura 2.

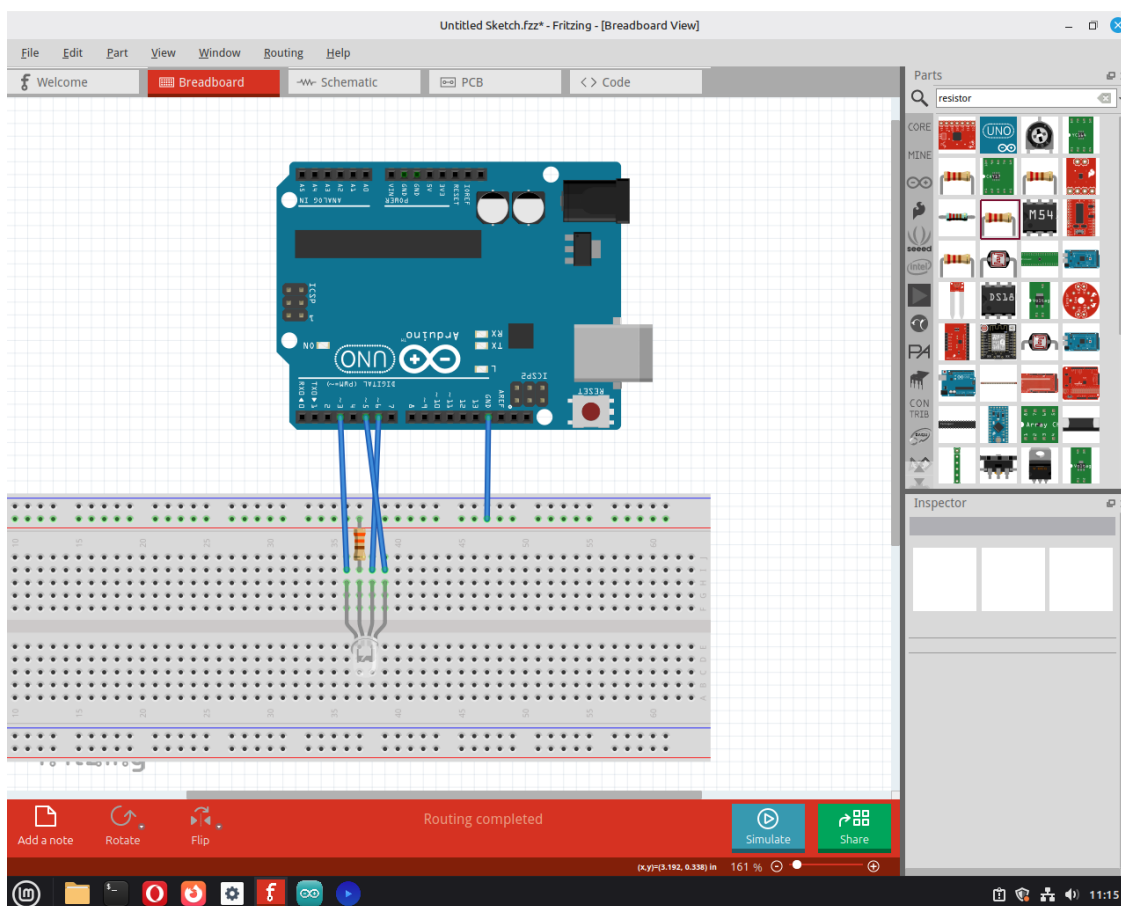


Figura 1: Diseño del circuito para el LED RGB en Fritzing.

La interfaz se hizo en Processing usando las librerías `controlP5` (para los controles de la pantalla) y `cc.arduino` (para hablar con la placa por Firmata). Lo primero fue declarar los pines y configurarlos como salida (Código 1):

Código 1: Declaración de pines y configuración en `setup()`.

```
int ledRojo = 3;
int ledVerde = 6;
int ledAzul = 5;

arduino = new Arduino(this, Arduino.list()[2], 57600);
```

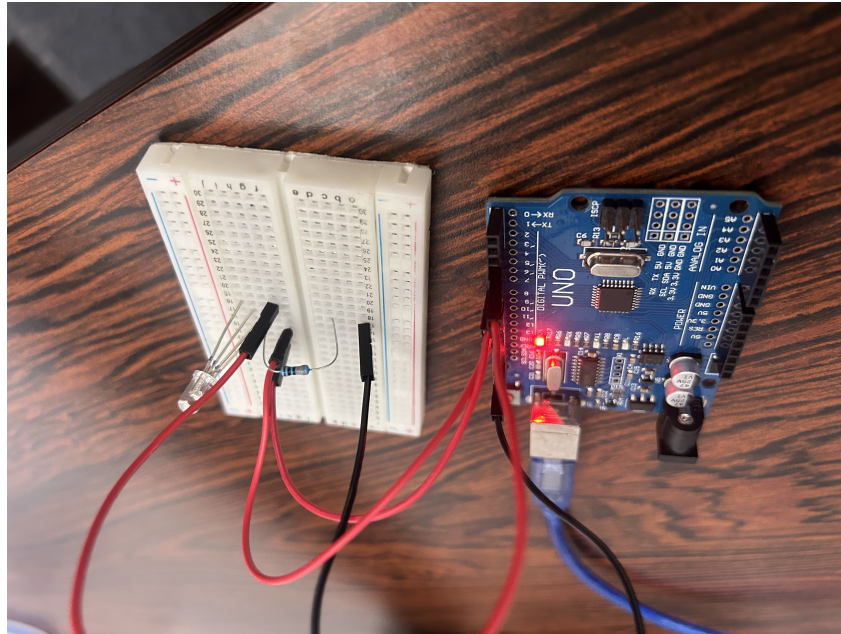


Figura 2: Montaje físico del LED RGB sobre el protoboard.

```
arduino.pinMode(ledRojo, Arduino.OUTPUT);  
arduino.pinMode(ledVerde, Arduino.OUTPUT);  
arduino.pinMode(ledAzul, Arduino.OUTPUT);
```

Los tres sliders se crearon usando `cp5.addSlider()` con rango de 0 a 255, que es el rango del PWM en 8 bits (Código 2). También se agregó un gráfico de línea para mostrar el historial de intensidad.

Código 2: Creación de sliders y gráfico en `setup()`.

```
cp5.addSlider("slider1").setPosition(50, 50).setRange(0, 255).setSize(150, 20);  
cp5.addSlider("slider2").setPosition(50, 110).setRange(0, 255).setSize(150, 20);  
cp5.addSlider("slider3").setPosition(50, 170).setRange(0, 255).setSize(150, 20);  
  
myChart1 = cp5.addChart("ledVerde")  
    .setPosition(50, 200).setSize(200, 100)  
    .setRange(0, 255).setView(Chart.LINE).setStrokeWeight(1.5);  
myChart1.addDataSet("incoming");  
myChart1.setData("incoming", new float[100]);
```

En `draw()` se lee el valor de cada slider y se manda al pin correspondiente con `analogWrite()`. También se llama a `dibujarLED()` para mostrar en pantalla el estado del LED, y se actualiza el gráfico cada 5 frames para que no sea tan pesado (Código 3). La función `dibujarLED()` usa el alfa del color para que el círculo en pantalla sea más o menos brillante según el valor del slider, igual que el LED físico (Código 4).

Código 3: Lectura de sliders, escritura en pines y actualización visual en `draw()`.

```
int valorSlider1 = (int)cp5.getController("slider1").getValue();  
int valorSlider2 = (int)cp5.getController("slider2").getValue();
```

```
int valorSlider3 = (int)cp5.getController("slider3").getValue();

arduino.analogWrite(ledRojo, valorSlider1);
arduino.analogWrite(ledVerde, valorSlider2);
arduino.analogWrite(ledAzul, valorSlider3);

dibujarLED(260, 60, color(255, 0, 0), valorSlider1);
dibujarLED(260, 120, color(0, 255, 0), valorSlider2);
dibujarLED(260, 180, color(0, 0, 255), valorSlider3);

if (frameCount % 5 == 0)
    myChart1.push("incoming", valorSlider3);
```

Código 4: Función para dibujar el LED virtual en pantalla.

```
void dibujarLED(int x, int y, color c, int val) {
    fill(red(c), green(c), blue(c), val);
    noStroke();
    ellipse(x, y, 40, 40);
    stroke(255, 50);
    noFill();
    ellipse(x, y, 30, 30);
}
```

Además se implementó un panel que muestra en tiempo real los valores de los tres canales y la fotorresistencia (Código 5).

Código 5: Panel de valores en tiempo real.

```
void dibujarVentanaValores(int x, int y, int w, int h) {
    fill(0, 0, 0, 150);
    stroke(255, 100);
    rect(x, y, w, h, 10);
    fill(0, 255, 0);
    textSize(12);
    text("DATOS EN TIEMPO REAL", x + 10, y + 20);
    fill(255);
    textSize(14);
    text("Led Rojo:      " + nf(cp5.getController("slider1").getValue(), 0, 1), x+10, y+45);
    text("Led Verde:     " + nf(cp5.getController("slider2").getValue(), 0, 1), x+10, y+65);
    text("Led Azul:      " + nf(cp5.getController("slider3").getValue(), 0, 1), x+10, y+85);
    text("Fotorresistencia: " + nf(arduino.analogRead(0), 0, 1), x+10, y+105);
}
```

En la Figura 3 se ve la interfaz corriendo con los sliders, los LEDs virtuales, el panel de valores y el gráfico. La Figura 4 muestra el LED físico encendido con el color que se le estaba mandando desde la interfaz.

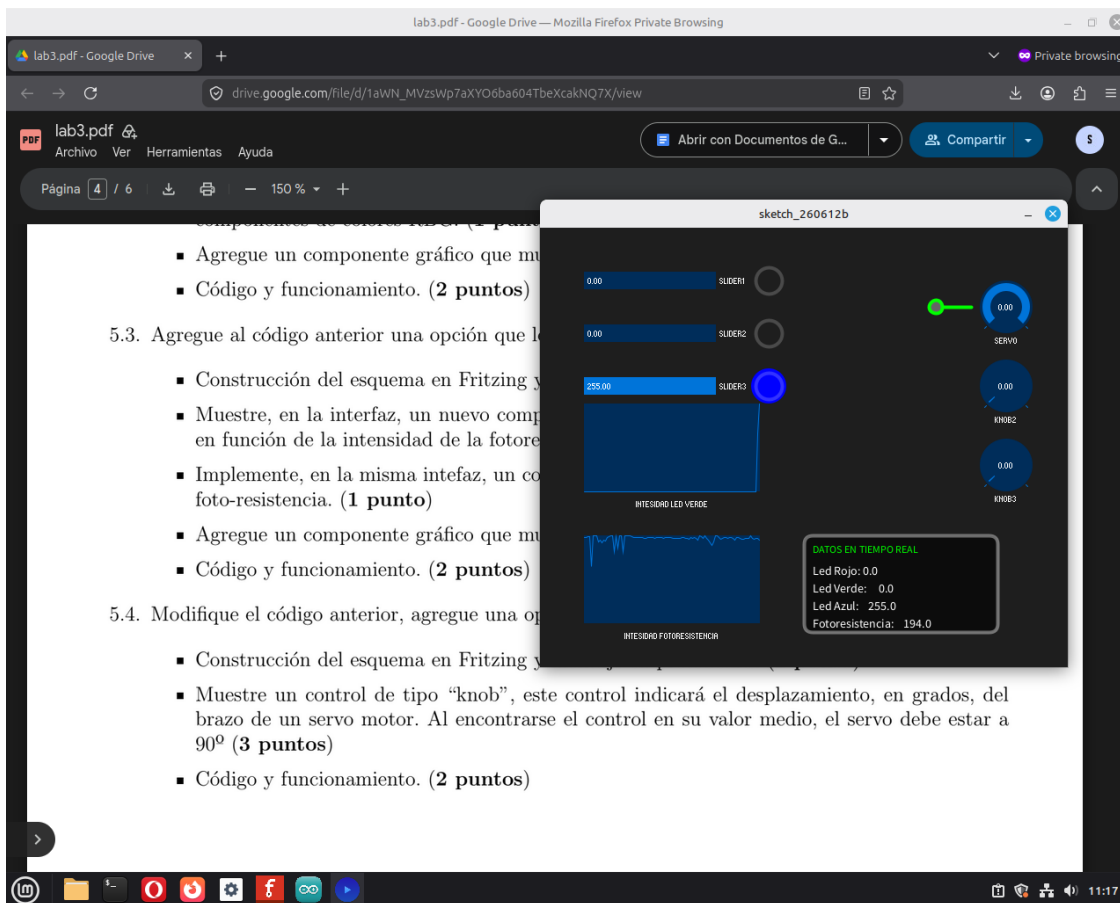


Figura 3: Interfaz en ejecución: sliders, LEDs virtuales, panel de valores y gráfico.

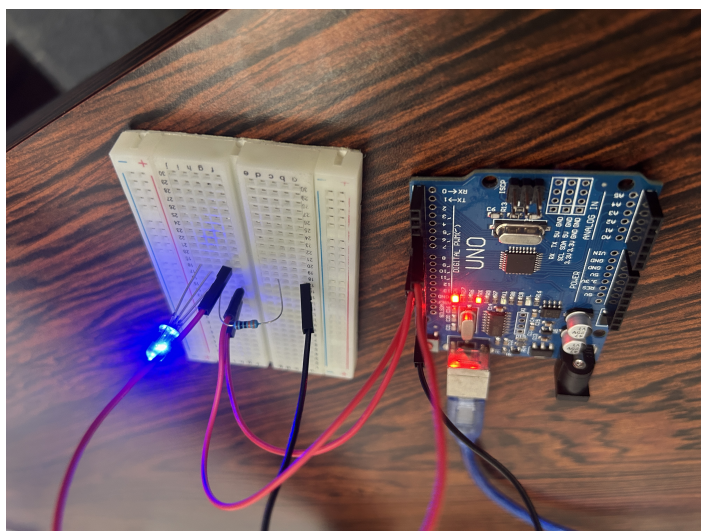


Figura 4: LED RGB encendido físicamente con el color definido en la interfaz.

1.3. Actividad 5.3: Lectura de fotorresistencia

Agregue al código anterior una opción que lea valores de una foto-resistencia.

1.4. Actividad 5.4: Control de servo motor

Enunciado: Modifique el código anterior, agregue una opción para controlar un servo motor. Muestre un control de tipo knob que indique el desplazamiento en grados del brazo del servo. Al encontrarse el control en su valor medio, el servo debe estar a 90°.

Se agregó un servo motor al circuito conectándolo al pin digital 10. Con Firmata, al configurar un pin como servo, la placa genera automáticamente la señal de posicionamiento sin que Processing tenga que hacer nada extra [1]. El diseño del circuito está en la Figura 5 y el montaje físico en la Figura 6.

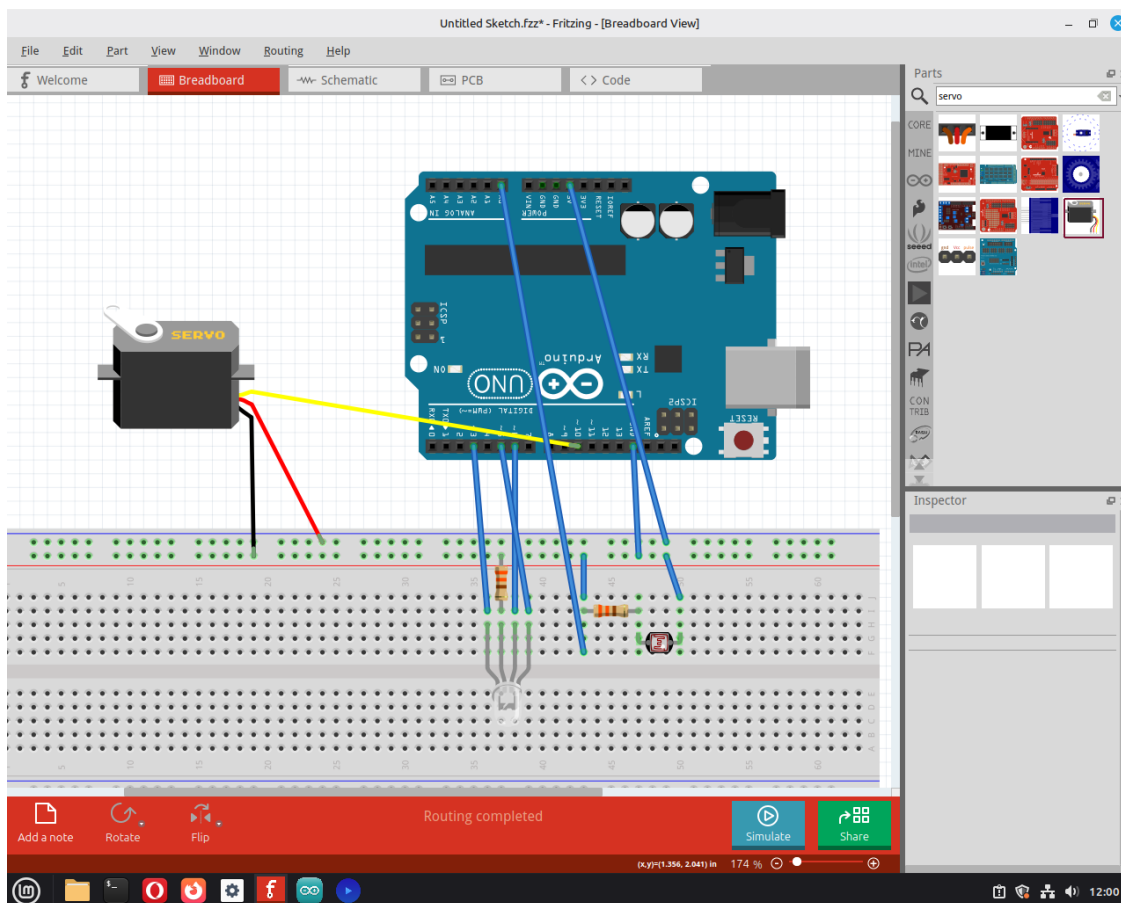


Figura 5: Diseño del circuito para el servo motor en Fritzing.

El pin 10 se configuró con `Arduino.SERVO` para que Firmata sepa que tiene que manejar ese pin como servo y no como salida digital normal (Código 6):

Código 6: Declaración y configuración del pin del servo.

```
int servoPin = 10;
...
arduino.pinMode(servoPin, Arduino.SERVO);
```

Para el knob se usó un rango de -180 a 0 en lugar de 0 a 180 (Código 7). Esto se hizo para que cuando el knob esté en la mitad su valor sea -90 , y al multiplicarlo por -1 antes de mandarlo al servo queda en 90° , que es justo lo que pide el enunciado. En `dibujarServo()` se dibuja una línea que rota igual

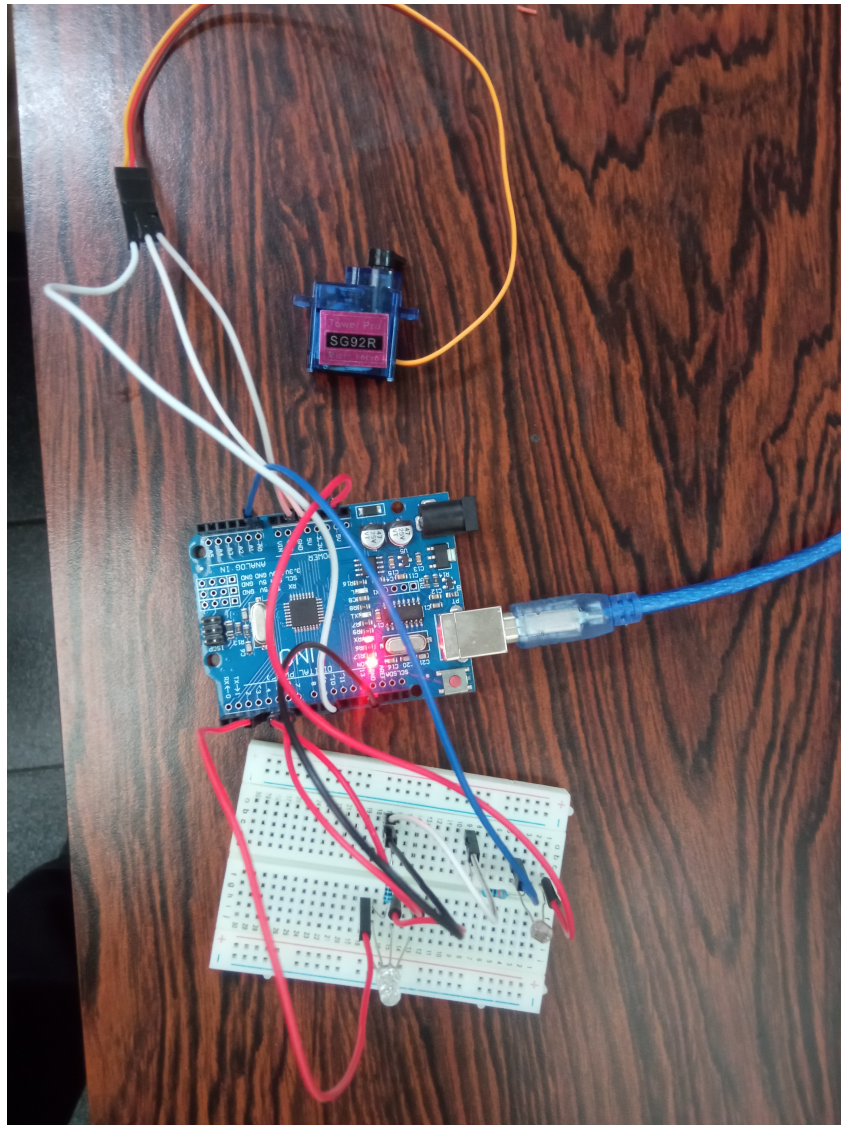


Figura 6: Montaje físico del servo motor sobre el protoboard.

que el brazo del servo físico.

Código 7: Knob del servo y su control en `draw()`.

```
// setup():
cp5.addKnob("Servo").setPosition(500, 60).setRadius(30).setRange(-180, 0);

// draw():
int anguloServo = (int)cp5.getController("Servo").getValue();
arduino.servoWrite(servoPin, -1 * anguloServo);
dibujarServo(anguloServo);
```

Código 8: Representación visual del brazo del servo en la interfaz.

```
void dibujarServo(int angulo) {
    pushMatrix();
```

```
translate(450, 90);  
rotate(radians(angulo));  
stroke(0, 255, 0);  
strokeWeight(4);  
line(0, 0, 40, 0);  
fill(100);  
ellipse(0, 0, 15, 15);  
popMatrix();  
}
```

La Figura 7 muestra la interfaz con el knob girado y el brazo virtual en la posición correspondiente. El servo físico respondió al mismo ángulo en tiempo real.

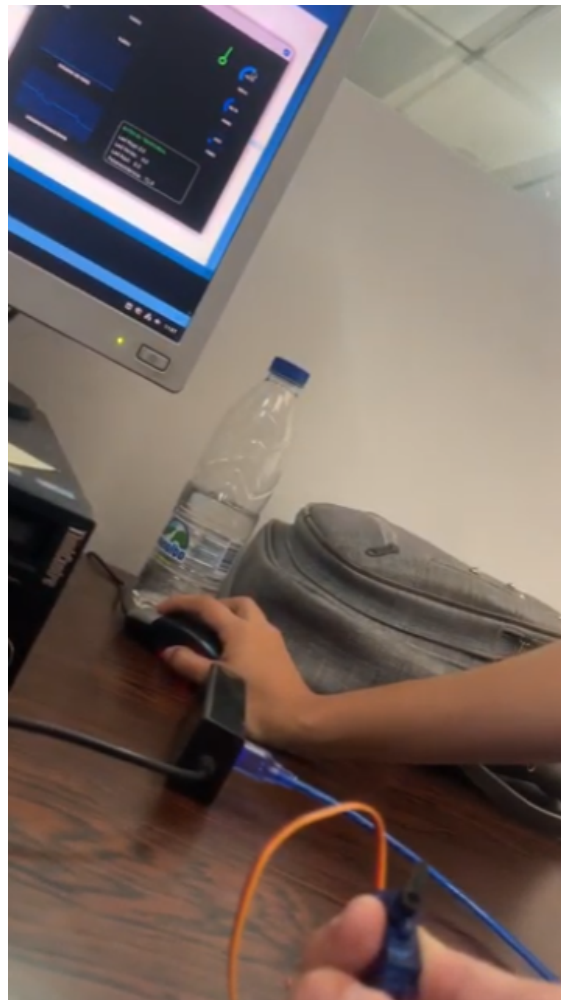


Figura 7: Interfaz con el knob activo y el servo respondiendo al mismo ángulo.

2. Conclusiones

Con estas actividades se pudo ver cómo Firmata simplifica mucho el trabajo, ya que no hay que reprogramar el Arduino cada vez que se quiere cambiar algo en la lógica del programa. Toda la parte de control queda en Processing y la placa solo obedece lo que se le manda por serial.

En la actividad 5.2 se comprobó que mezclar colores en el LED RGB es directamente proporcional al valor PWM de cada canal. El detalle de usar el alfa del color en los LEDs virtuales funcionó bien para representar el brillo en pantalla de la misma forma que en el físico.

En la actividad 5.4 el truco del rango negativo en el knob (-180 a 0) fue lo que permitió cumplir el requisito de que el centro coincidiera con 90° en el servo, simplemente negando el valor antes de mandarlo con `servoWrite()` [2].

Referencias

- [1] F. Mellis, H. Barragan y otros, “Firmata Protocol Documentation,” *Firmata GitHub Repository*, 2006–2024. [En línea]. Disponible: <https://github.com/firmata/protocol>. [Consulta: jun. 2026]
- [2] Arduino, “Arduino Language Reference,” *Arduino Documentation*, Arduino LLC, 2024. [En línea]. Disponible: <https://docs.arduino.cc/language-reference/>. [Consulta: jun. 2026]
- [3] A. Cockpit, “controlP5 — A GUI Library for Processing,” 2012–2023. [En línea]. Disponible: <https://www.sojamo.de/libraries/controlP5/>. [Consulta: jun. 2026]
- [4] A. Russoniello, “Laboratorio: Interfaz Gráfica con Processing y Protocolo Firmata,” guía de práctica de laboratorio, Laboratorio General (Internet de las Cosas), Escuela de Computación, Facultad de Ciencias, Universidad Central de Venezuela, Caracas, 2026.